StorageGRID에서 S3 REST API를 구현하는 방법

StorageGRID 11.7

NetApp
June 09, 2023

This PDF was generated from https://docs.netapp.com/ko-kr/storagegrid-117/s3/conflicting-client-requests.html on June 09, 2023. Always check docs.netapp.com for the latest.

목차

StorageGRID에서 S3 REST API를 구현하는 방법 . . . 1
클라이언트 요청 충돌 . . . 1
일관성 제어 . . . 1
StorageGRID ILM 규칙이 개체를 관리하는 방법 . . . 3
오브젝트 버전 관리 . . . 5
S3 REST API를 사용하여 S3 오브젝트 잠금을 구성합니다 . . . 5
S3 라이프사이클 구성을 생성합니다 . . . 11
S3 REST API 구현을 위한 권장사항 . . . 15

StorageGRID에서 S3 REST API를 구현하는 방법

클라이언트 요청 충돌

동일한 키에 쓰는 두 클라이언트 등의 충돌하는 클라이언트 요청은 "최신 성공" 기준으로 해결됩니다.

"Latest-WINS" 평가 시기는 S3 클라이언트가 작업을 시작할 때가 아니라 StorageGRID 시스템이 지정된 요청을 완료하는 시점을 기준으로 합니다.

일관성 제어

일관성 제어는 애플리케이션의 요구에 따라 오브젝트의 가용성과 서로 다른 스토리지 노드 및 사이트 전체에서 오브젝트의 일관성 간의 균형을 제공합니다.

기본적으로 StorageGRID는 새로 생성된 개체에 대해 쓰기 후 읽기 일관성을 보장합니다. 성공적으로 완료된 PUT를 팔로우하면 새로 작성된 데이터를 읽을 수 있습니다. 기존 오브젝트, 메타데이터 업데이트 및 삭제를 덮어쓰는 것은 결국 일관성이 유지됩니다. 덮어쓰기는 일반적으로 전파되는 데 몇 초 또는 몇 분이 걸리지만 최대 15일이 소요될 수 있습니다.

오브젝트 작업을 다른 정합성 보장 레벨에서 수행하려는 경우 각 버킷 또는 각 API 작업에 대해 정합성 제어를 지정할 수 있습니다.

일관성 제어

정합성 보장 제어는 StorageGRID에서 객체를 추적하는 데 사용하는 메타데이터가 노드 간에 분산되므로 클라이언트 요청에 대한 객체의 가용성에 영향을 줍니다.

버킷 또는 API 작업에 대한 정합성 제어를 다음 값 중 하나로 설정할 수 있습니다.

- * ALL *: 모든 노드가 즉시 데이터를 수신하거나 요청이 실패합니다.
- * strong-global *: 모든 사이트에서 모든 클라이언트 요청에 대해 쓰기 후 읽기 일관성을 보장합니다.
- * strong-site *: 사이트 내의 모든 클라이언트 요청에 대해 쓰기 후 읽기 일관성을 보장합니다.
- * read-after-new-write *: (기본값)는 새 객체에 대한 읽기 후 쓰기 정합성을 보장하고 객체 업데이트에 대한 최종 일관성을 제공합니다. 고가용성 및 데이터 보호 보장 제공 대부분의 경우에 권장됩니다.
- * 사용 가능 *: 새 객체 및 객체 업데이트 모두에 대한 최종 일관성을 제공합니다. S3 버킷의 경우 필요한 경우에만 사용하십시오(예: 거의 읽지 않는 로그 값이 포함된 버킷의 경우 또는 존재하지 않는 키의 헤드 또는 GET 작업의 경우). S3 FabricPool 버킷은 지원되지 않습니다.

"새 쓰기 후"와 "사용 가능한" 일관성 제어 기능을 사용합니다

HEAD 또는 GET 연산에서 "read-after-new-write" 정합성 제어를 사용하면 StorageGRID는 다음과 같이 여러 단계로 조회를 수행합니다.

- 먼저 낮은 일관성을 사용하여 오브젝트를 찾습니다.
- 이 조회에 실패하면 강력한 글로벌 동작에 해당하는 일관성 수준에 도달할 때까지 조회가 다음 일관성 수준에서

반복됩니다.

헤드 또는 GET 연산에서 "READ-After-NEW-WRITE" 일관성 컨트롤을 사용하지만 개체가 없으면 개체 조회는 항상 강력한 글로벌 동작의 일관성 수준에 도달합니다. 이 정합성 보장 수준에서는 각 사이트에서 개체 메타데이터의 여러 복사본을 사용할 수 있어야 하므로 같은 사이트에 있는 둘 이상의 스토리지 노드를 사용할 수 없는 경우 500개의 내부 서버 오류가 많이 발생할 수 있습니다.

Amazon S3와 유사한 일관성 보증이 필요하지 않으면 일관성 제어를 ""사용 가능""으로 설정하여 헤드 및 GET 작업에 대한 이러한 오류를 방지할 수 있습니다. 헤드 또는 GET 작업에서 "사용 가능한" 정합성 제어를 사용할 경우 StorageGRID는 최종 일관성만 제공합니다. 일관성 수준을 높일 때 실패한 작업을 다시 시도하지 않으므로 오브젝트 메타데이터의 여러 복사본을 사용할 필요가 없습니다.

API 작업에 대한 정합성 제어를 지정합니다

개별 API 작업의 정합성 제어를 설정하려면 작업에 대해 정합성 보장 제어가 지원되어야 하며 요청 헤더에 정합성 제어를 지정해야 합니다. 이 예제에서는 개체 가져오기 작업을 위해 일관성 컨트롤을 "문자열 사이트"로 설정합니다.

```
GET /bucket/object HTTP/1.1
Date: date
Authorization: authorization name
Host: host
Consistency-Control: strong-site
```

개체 넣기 작업과 개체 가져오기 작업 모두에 대해 동일한 일관성 컨트롤을 사용해야 합니다.

버킷의 정합성 제어를 지정합니다

버킷의 일관성 제어를 설정하려면 StorageGRID PUT 버킷 정합성 보장 요청 및 GET 버킷 정합성 보장 요청을 사용할 수 있습니다. 또는 테넌트 관리자 또는 테넌트 관리 API를 사용할 수 있습니다.

버킷의 정합성 제어 기능을 설정할 때는 다음 사항에 유의하십시오.

- 버킷의 일관성 제어를 설정하면 버킷의 오브젝트 또는 버킷 구성에 대해 수행된 S3 작업에 사용되는 일관성 제어가 결정됩니다. 버킷 자체의 작동에는 영향을 미치지 않습니다.
- 개별 API 작업의 정합성 제어는 버킷의 정합성 제어를 재정의합니다.
- 일반적으로 버킷은 기본 일관성 제어인 "read-after-new-write"를 사용해야 합니다. 요청이 올바르게 작동하지 않는 경우 가능한 경우 응용 프로그램 클라이언트 동작을 변경합니다. 또는 클라이언트가 각 API 요청에 대한 정합성 제어를 지정하도록 구성합니다. 버킷 레벨에서만 정합성 제어를 최후의 수단으로 설정하십시오.

정합성 보장 제어와 ILM 규칙이 상호 작용하여 데이터 보호에 영향을 미치는 방식

일관성 제어와 ILM 규칙 모두 오브젝트의 보호 방법에 영향을 미칩니다. 이러한 설정은 상호 작용할 수 있습니다.

예를 들어, 개체가 저장될 때 사용되는 일관성 컨트롤은 오브젝트 메타데이터의 초기 배치에 영향을 미치는 반면 ILM 규칙에 대해 선택된 수집 동작은 오브젝트 복사본의 초기 배치에 영향을 줍니다. StorageGRID에서는 클라이언트 요청을 이행하기 위해 오브젝트의 메타데이터와 해당 데이터에 모두 액세스해야 하므로 일관성 수준과 수집 동작에 적합한 보호 수준을 선택하면 초기 데이터 보호 수준을 높이고 시스템 응답을 더욱 정확하게 예측할 수 있습니다.

ILM 규칙에 대해 다음과 같은 수집 동작을 사용할 수 있습니다.

- * 이중 커밋 *: StorageGRID은 즉시 개체의 임시 복사본을 만들고 클라이언트에 성공을 반환합니다. ILM 규칙에 지정된 복사본은 가능한 경우 만들어집니다.
- * Strict * : ILM 규칙에 지정된 모든 사본은 클라이언트에 반환되기 전에 만들어야 합니다.
- * 균형 *: StorageGRID는 수집 시 ILM 규칙에 지정된 모든 복제본을 생성하려고 합니다. 그렇지 않을 경우 중간 복사본이 만들어지고 클라이언트에 성공적으로 반환됩니다. ILM 규칙에 지정된 복사본은 가능한 경우 만들어집니다.

> ILM 규칙의 수집 동작을 선택하기 전에 정보 수명 주기 관리를 통해 개체를 관리하기 위한 지침에서 이러한 설정에 대한 전체 설명을 읽어보십시오.

일관성 제어 및 **ILM** 규칙이 상호 작용하는 방법의 예

다음 ILM 규칙 및 다음 일관성 수준 설정이 있는 두 사이트 그리드가 있다고 가정합니다.

- * ILM 규칙 *: 로컬 사이트와 원격 사이트에 각각 하나씩, 두 개의 오브젝트 복사본을 만듭니다. Strict 수집 동작이 선택됩니다.
- * Consistency level *: "trong-global"(개체 메타데이터가 모든 사이트에 즉시 배포됩니다.)

클라이언트가 오브젝트를 그리드에 저장할 때 StorageGRID는 오브젝트 복사본을 둘 다 만들고 메타데이터를 두 사이트에 분산한 다음 클라이언트에 성공을 반환합니다.

수집 성공 메시지가 표시된 시점에 객체가 손실로부터 완벽하게 보호됩니다. 예를 들어, 수집 직후 로컬 사이트가 손실되면 오브젝트 데이터와 오브젝트 메타데이터의 복사본이 원격 사이트에 계속 존재합니다. 개체를 완전히 검색할 수 있습니다.

대신 동일한 ILM 규칙 및 "'strong-site' 정합성 보장 수준을 사용한 경우 객체 데이터가 원격 사이트에 복제되었지만 객체 메타데이터가 그 위치에 배포되기 전에 클라이언트에 성공 메시지가 표시될 수 있습니다. 이 경우 오브젝트 메타데이터의 보호 수준이 오브젝트 데이터의 보호 수준과 일치하지 않습니다. 수집 후 곧바로 로컬 사이트가 손실되면 오브젝트 메타데이터가 손실됩니다. 개체를 검색할 수 없습니다.

일관성 수준과 ILM 규칙 간의 상호 관계는 복잡할 수 있습니다. 도움이 필요한 경우 NetApp에 문의하십시오.

관련 정보

"ILM을 사용하여 개체를 관리합니다"

"버킷 일관성 확보"

"버킷 일관성을 유지합니다"

StorageGRID ILM 규칙이 개체를 관리하는 방법

그리드 관리자는 정보 라이프사이클 관리(ILM) 규칙을 생성하여 S3 REST API 클라이언트 애플리케이션에서 StorageGRID 시스템으로 수집된 오브젝트 데이터를 관리합니다. 그런 다음 이러한 규칙을 ILM 정책에 추가하여 시간 경과에 따라 오브젝트 데이터가 저장되는 방법 및 위치를 결정합니다.

ILM 설정은 개체의 다음 측면을 결정합니다.

- * 지역 *

  StorageGRID 시스템(스토리지 풀) 또는 클라우드 스토리지 풀 내에서 오브젝트 데이터의 위치입니다.

- * 스토리지 등급 *

  오브젝트 데이터를 저장하는 데 사용되는 스토리지의 유형(예: 플래시 또는 회전식 디스크)

- * 손실 방지 *

  복제, 삭제 코딩 또는 두 가지 유형의 복사본을 만들 수와 복사본 유형을 지정합니다.

- * 보존 *

  오브젝트의 데이터 관리 방식, 저장 위치 및 데이터 손실을 보호하는 방법에 대한 시간이 지나면서 변동합니다.

- * 수집 중 보호 *

  수집 중에 오브젝트 데이터를 보호하는 데 사용되는 방법: 동기 배치(Ingest 동작에 대한 균형 또는 엄격 옵션 사용) 또는 중간 복사본 만들기(이중 커밋 옵션 사용).

ILM 규칙을 사용하여 개체를 필터링 및 선택할 수 있습니다. S3을 사용하여 수집된 개체의 경우 ILM 규칙을 통해 다음 메타데이터를 기반으로 개체를 필터링할 수 있습니다.

- 테넌트 계정입니다
- 버킷 이름
- 수집 시간
- 키
- 마지막 액세스 시간입니다

> 기본적으로 마지막 액세스 시간에 대한 업데이트는 모든 S3 버킷에 대해 비활성화됩니다. StorageGRID 시스템에 마지막 액세스 시간 옵션을 사용하는 ILM 규칙이 포함된 경우 해당 규칙에 지정된 S3 버킷의 마지막 액세스 시간에 대한 업데이트를 활성화해야 합니다. Put Bucket Last Access Time 요청인 Tenant Manager를 사용합니다(참조) "마지막 액세스 시간 업데이트를 사용하거나 사용하지 않도록 설정합니다") 또는 테넌트 관리 API. 마지막 액세스 시간 업데이트를 설정할 때는 특히 작은 오브젝트가 있는 시스템에서 StorageGRID 성능이 저하될 수 있다는 점에 유의하십시오.

- 위치 제약 조건
- 개체 크기
- 사용자 메타데이터
- 개체 태그

관련 정보

"테넌트 계정을 사용합니다"

"ILM을 사용하여 개체를 관리합니다"

"버킷 최종 접근 시간"

오브젝트 버전 관리

버전 관리를 사용하면 개체의 여러 버전을 유지하여 실수에 의한 개체 삭제로부터 보호하고 이전 버전의 개체를 검색하고 복원할 수 있습니다.

StorageGRID 시스템은 대부분의 기능을 지원하는 버전 관리를 구현하지만 몇 가지 제한 사항이 있습니다. StorageGRID는 각 오브젝트의 버전을 최대 1,000개까지 지원합니다.

오브젝트 버전 관리를 StorageGRID ILM(정보 라이프사이클 관리) 또는 S3 버킷 라이프사이클 구성과 결합할 수 있습니다. 버킷에 대해 이 기능을 설정하려면 각 버킷에 대해 버전 관리를 명시적으로 활성화해야 합니다. 버킷의 각 오브젝트에는 StorageGRID 시스템에서 생성되는 버전 ID가 할당됩니다.

MFA(다중 요소 인증) 삭제 사용은 지원되지 않습니다.

> 버전 관리는 StorageGRID 버전 10.3 이상으로 생성된 버킷에서만 사용할 수 있습니다.

ILM 및 버전 관리

ILM 정책은 개체의 각 버전에 적용됩니다. ILM 스캔 프로세스는 모든 개체를 지속적으로 스캔하고 현재 ILM 정책에 대해 다시 평가합니다. ILM 정책에 대한 모든 변경 사항은 이전에 수집된 모든 개체에 적용됩니다. 여기에는 버전 관리가 활성화된 경우 이전에 수집된 버전이 포함됩니다. ILM 스캐닝은 이전에 수집된 개체에 새로운 ILM 변경 사항을 적용합니다.

버전 관리가 활성화된 버킷의 S3 오브젝트에서 버전 관리를 지원하므로 ""비현재 시간""을 참조 시간으로 사용하는 ILM 규칙을 만들 수 있습니다("이 규칙을 이전 개체 버전에만 적용하시겠습니까?" 질문에 대해 * 예 * 를 선택합니다." 인치 "ILM 규칙 만들기 마법사의 1단계")를 클릭합니다. 개체가 업데이트되면 이전 버전은 업데이트되지 않습니다. ""비현재 시간"" 필터를 사용하면 이전 버전의 오브젝트에 대한 스토리지 영향을 줄이는 정책을 생성할 수 있습니다.

> 다중 파트 업로드 작업을 사용하여 새 버전의 개체를 업로드할 때 개체의 원래 버전에 대한 비현재 시간은 다중 파트 업로드가 완료될 때가 아닌 새 버전에 대해 다중 파트 업로드가 생성된 시점을 반영합니다. 제한된 경우 원래 버전의 비현재 시간이 현재 버전의 시간보다 몇 시간 또는 며칠 빨라질 수 있습니다.

을 참조하십시오 "S3 버전 오브젝트 ILM 규칙 및 정책(예 4)".

S3 REST API를 사용하여 S3 오브젝트 잠금을 구성합니다

StorageGRID 시스템에서 글로벌 S3 오브젝트 잠금 설정이 활성화된 경우 S3 오브젝트 잠금이 활성화된 버킷을 생성할 수 있습니다. 각 오브젝트 버전에 대해 각 버킷의 기본 보존 또는 보존 설정을 지정할 수 있습니다.

버킷에 대해 S3 오브젝트 잠금을 활성화하는 방법

StorageGRID 시스템에 대해 글로벌 S3 오브젝트 잠금 설정이 활성화된 경우 각 버킷을 생성할 때 선택적으로 S3 오브젝트 잠금을 활성화할 수 있습니다.

S3 오브젝트 잠금은 버킷을 생성할 때만 활성화할 수 있는 영구 설정입니다. 버킷을 생성한 후에는 S3 오브젝트 잠금을

추가하거나 비활성화할 수 없습니다.

버킷에 대해 S3 오브젝트 잠금을 설정하려면 다음 방법 중 하나를 사용하십시오.

- 테넌트 관리자를 사용하여 버킷을 생성합니다. 을 참조하십시오 "S3 버킷을 생성합니다".
- 과 함께 PUT 버킷 요청을 사용하여 버킷을 작성합니다 `x-amz-bucket-object-lock-enabled` 요청 헤더. 을 참조하십시오 "버킷 작업".

S3 오브젝트 잠금에서는 버킷 버전 관리가 필요하며, 이 버전은 버킷을 생성할 때 자동으로 활성화됩니다. 버킷의 버전 관리는 일시 중단할 수 없습니다. 을 참조하십시오 "오브젝트 버전 관리".

버킷의 기본 보존 설정입니다

버킷에 대해 S3 오브젝트 잠금이 활성화된 경우 버킷에 대한 기본 보존을 선택적으로 설정하고 기본 보존 모드 및 기본 보존 기간을 지정할 수 있습니다.

기본 보존 모드

- 규정 준수 모드:
  - 보존 기한 에 도달할 때까지 개체를 삭제할 수 없습니다.
  - 오브젝트의 보존 기한 을 늘릴 수 있지만 줄일 수는 없습니다.
  - 개체의 보존 기한 은 해당 날짜에 도달할 때까지 제거할 수 없습니다.
- 거버넌스 모드:
  - 를 가진 사용자 `s3:BypassGovernanceRetention` 사용 권한은 를 사용할 수 있습니다 `x-amz-bypass-governance-retention: true` 보존 설정을 무시하도록 헤더를 요청합니다.
  - 이러한 사용자는 보존 기한이 되기 전에 개체 버전을 삭제할 수 있습니다.
  - 이러한 사용자는 개체의 보존 기간(Retain-until-date)을 증가, 감소 또는 제거할 수 있습니다.

기본 보존 기간

각 버킷에는 년 또는 일 단위로 지정된 기본 보존 기간이 있을 수 있습니다.

버킷의 기본 보존 설정 방법

버킷의 기본 보존을 설정하려면 다음 방법 중 하나를 사용합니다.

- 테넌트 관리자에서 버킷 설정을 관리합니다. 을 참조하십시오 "S3 버킷을 생성합니다" 및 "S3 오브젝트 잠금 기본 보존 업데이트".
- 버킷에 대한 Put Object Lock Configuration 요청을 발행하여 기본 모드 및 기본 일/년 수를 지정합니다.

개체 잠금 구성을 배치합니다

Put Object Lock Configuration 요청을 사용하면 S3 Object Lock이 활성화된 버킷의 기본 보존 모드 및 기본 보존 기간을 설정 및 수정할 수 있습니다. 이전에 구성한 기본 보존 설정을 제거할 수도 있습니다.

새 오브젝트 버전이 버킷에 수집되면 기본 보존 모드가 적용됩니다(인 경우) `x-amz-object-lock-mode` 및 `x-amz-object-lock-retain-until-date` 지정되지 않았습니다. 기본 보존 기간은 다음 경우에 보존 기간을

계산하는 데 사용됩니다 `x-amz-object-lock-retain-until-date` 이(가) 지정되지 않았습니다.

오브젝트 버전을 수집한 후 기본 보존 기간을 수정하면 오브젝트 버전의 보존 기한은 그대로 유지되고 새 기본 보존 기간을 사용하여 다시 계산되지 않습니다.

에 가 있어야 합니다 `s3:PutBucketObjectLockConfiguration` 이 작업을 완료하려면 권한 또는 계정 루트 권한이 있어야 합니다.

를 클릭합니다 `Content-MD5` 요청 헤더를 PUT 요청에 지정해야 합니다.

요청 예

이 예에서는 버킷에 대해 S3 Object Lock을 설정하고 기본 보존 모드를 규정 준수 로 설정하고 기본 보존 기간을 6년으로 설정합니다.

```
PUT /bucket?object-lock HTTP/1.1
Accept-Encoding: identity
Content-Length: 308
Host: host
Content-MD5: request header
User-Agent: s3sign/1.0.0 requests/2.24.0 python/3.8.2
X-Amz-Date: date
X-Amz-Content-SHA256: authorization-string
Authorization: authorization-string

<ObjectLockConfiguration>
    <ObjectLockEnabled>Enabled</ObjectLockEnabled>
    <Rule>
        <DefaultRetention>
            <Mode>COMPLIANCE</Mode>
            <Years>6</Years>
        </DefaultRetention>
    </Rule>
</ObjectLockConfiguration>
```

버킷의 기본 보존 결정 방법

버킷에 대해 S3 오브젝트 잠금이 설정되었는지 확인하고 기본 보존 모드 및 보존 기간을 확인하려면 다음 방법 중 하나를 사용하십시오.

- 테넌트 관리자에서 버킷을 확인합니다. 을 참조하십시오 "S3 버킷을 봅니다".
- 객체 잠금 구성 가져오기 요청을 발행합니다.

개체 잠금 구성을 가져옵니다

객체 잠금 구성 가져오기 요청을 사용하면 버킷에 대해 S3 오브젝트 잠금이 설정되어 있는지 확인하고, 활성화된 경우 버킷에 대해 기본 보존 모드 및 보존 기간이 구성되어 있는지 확인할 수 있습니다.

새 오브젝트 버전이 버킷에 수집되면 기본 보존 모드가 적용됩니다(인 경우) `x-amz-object-lock-mode` 이(가) 지정되지 않았습니다. 기본 보존 기간은 다음 경우에 보존 기간을 계산하는 데 사용됩니다 `x-amz-object-lock-retain-until-date` 이(가) 지정되지 않았습니다.

에 가 있어야 합니다 `s3:GetBucketObjectLockConfiguration` 이 작업을 완료하려면 권한 또는 계정 루트 권한이 있어야 합니다.

요청 예

```
GET /bucket?object-lock HTTP/1.1
Host: host
Accept-Encoding: identity
User-Agent: aws-cli/1.18.106 Python/3.8.2 Linux/4.4.0-18362-Microsoft
botocore/1.17.29
x-amz-date: date
x-amz-content-sha256: authorization-string
Authorization: authorization-string
```

응답 예

```
HTTP/1.1 200 OK
x-amz-id-2:
iVmcB7OXXJRkRH1FiVq1151/T24gRfpwpuZrEG11Bb9ImOMAAe98oxSpXlknabA0LTvBYJpSIX
k=
x-amz-request-id: B34E94CACB2CEF6D
Date: Fri, 04 Sep 2020 22:47:09 GMT
Transfer-Encoding: chunked
Server: AmazonS3

<?xml version="1.0" encoding="UTF-8"?>
<ObjectLockConfiguration xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
    <ObjectLockEnabled>Enabled</ObjectLockEnabled>
    <Rule>
        <DefaultRetention>
            <Mode>COMPLIANCE</Mode>
            <Years>6</Years>
        </DefaultRetention>
    </Rule>
</ObjectLockConfiguration>
```

개체의 보존 설정을 지정하는 방법

S3 오브젝트 잠금이 활성화된 버킷에는 S3 오브젝트 잠금 보존 설정이 있는 오브젝트와 없는 오브젝트의 조합이 포함될 수 있습니다.

오브젝트 레벨의 보존 설정은 S3 REST API를 사용하여 지정됩니다. 객체에 대한 보존 설정은 버킷의 기본 보존 설정보다 우선합니다.

각 개체에 대해 다음 설정을 지정할 수 있습니다.

- * 보존 모드 *: 규정 준수 또는 거버넌스 중 하나입니다.
- * Retain-until-date *: StorageGRID에서 개체 버전을 유지해야 하는 기간을 지정하는 날짜입니다.
  - 준수 모드에서 보존 기한이 미래인 경우 오브젝트를 검색할 수 있지만 수정하거나 삭제할 수 없습니다. 보관 기한 을 늘릴 수 있지만 이 날짜는 감소 또는 제거할 수 없습니다.
  - 거버넌스 모드에서 특별 권한이 있는 사용자는 보존 기한 설정을 무시할 수 있습니다. 보존 기간이 경과하기 전에 객체 버전을 삭제할 수 있습니다. 또한 보존 기간을 늘리거나 줄이거나 제거할 수도 있습니다.
- * 법적 증거 자료 보관 *: 개체 버전에 법적 증거 자료 보관 기능을 적용하면 해당 개체가 즉시 잠깁니다. 예를 들어 조사 또는 법적 분쟁과 관련된 객체에 법적 보류를 지정해야 할 수 있습니다. 법적 보류는 만료 날짜가 없지만 명시적으로 제거될 때까지 유지됩니다.

  개체에 대한 법적 보류 설정은 보존 모드 및 보존 기한 과 무관합니다. 개체 버전이 법적 증거 자료 보관 중인 경우 해당 버전을 삭제할 수 없습니다.

오브젝트 버전을 버킷에 추가할 때 S3 오브젝트 잠금 설정을 지정하려면 을 실행합니다 "개체 를 넣습니다", "개체 - 복사를 선택합니다", 또는 "멀티파트 업로드를 시작합니다" 요청하십시오.

다음을 사용할 수 있습니다.

- `x-amz-object-lock-mode`규정 준수 또는 거버넌스(대/소문자 구분)일 수 있습니다.

  > 를 지정할 경우 `x-amz-object-lock-mode`, 또한 을 지정해야 합니다 `x-amz-object-lock-retain-until-date`.

- `x-amz-object-lock-retain-until-date`
  - 보존 기간 값은 형식이어야 합니다 `2020-08-10T21:46:00Z`. 소수 자릿수는 허용되지만 소수점 이하 자릿수는 3자리만 유지됩니다(밀리초 단위). 다른 ISO 8601 형식은 허용되지 않습니다.
  - 보존 종료 날짜는 미래여야 합니다.
- `x-amz-object-lock-legal-hold`

  법적 증거 자료 보관(대소문자 구분)이 켜져 있는 경우, 해당 물체는 법적 증거 자료 보관 하에 배치됩니다. 법적 증거 자료 보관 기능이 꺼져 있는 경우 법적 증거 자료 보관 작업이 없습니다. 다른 값을 사용하면 400개의 잘못된 요청(InvalidArgument) 오류가 발생합니다.

이러한 요청 헤더를 사용하는 경우 다음과 같은 제한 사항에 유의하십시오.

- 를 클릭합니다 `Content-MD5` 요청 헤더가 필요한 경우 `x-amz-object-lock-*` 요청 헤더가 Put Object 요청에 있습니다. `Content-MD5` Put Object(개체 저장) - Copy(복사) 또는 Initiate MultiPart Upload(다중 파트 업로드)에는 필요하지 않습니다.
- 버킷에 S3 오브젝트 잠금이 설정되어 있지 않은 경우 및 가 활성화되어 있어야 합니다 `x-amz-object-lock-*` 요청 헤더가 있으면 400개의 잘못된 요청(InvalidRequest) 오류가 반환됩니다.
- Put Object 요청은 의 사용을 지원합니다 `x-amz-storage-class: REDUCED_REDUNDANCY` AWS 동작과 일치시킵니다. 하지만 오브젝트가 S3 오브젝트 잠금이 설정된 버킷으로 수집되면 StorageGRID는 항상 이중 커밋

수집을 수행합니다.

- 후속 Get 또는 Head Object 버전 응답에는 헤더가 포함됩니다 `x-amz-object-lock-mode`, `x-amz-object-lock-retain-until-date`, 및 `x-amz-object-lock-legal-hold`, 구성된 경우 및 요청 보낸 사람이 올바른 경우 `s3:Get*` 권한.

를 사용할 수 있습니다 `s3:object-lock-remaining-retention-days` 개체에 대해 허용되는 최소 및 최대 보존 기간을 제한하는 정책 조건 키입니다.

개체의 보존 설정을 업데이트하는 방법

기존 개체 버전에 대한 법적 증거 자료 보관 또는 보존 설정을 업데이트해야 하는 경우 다음 개체 하위 리소스 작업을 수행할 수 있습니다.

- `PUT Object legal-hold`

  새 법적 증거 자료 보관 값이 켜져 있으면 해당 개체는 법적 증거 자료 보관 아래에 배치됩니다. 법적 증거 자료 보관 가치가 없는 경우 법적 구속이 해제됩니다.

- `PUT Object retention`
  - 모드 값은 규정 준수 또는 거버넌스(대/소문자 구분)일 수 있습니다.
  - 보존 기간 값은 형식이어야 합니다 `2020-08-10T21:46:00Z`. 소수 자릿수는 허용되지만 소수점 이하 자릿수는 3자리만 유지됩니다(밀리초 단위). 다른 ISO 8601 형식은 허용되지 않습니다.
  - 개체 버전에 기존 보존 기한이 있는 경우 개체 버전을 늘릴 수만 있습니다. 새 값은 미래여야 합니다.

거버넌스 모드 사용 방법

를 가진 사용자 `s3:BypassGovernanceRetention` 권한은 거버넌스 모드를 사용하는 개체의 활성 보존 설정을 무시할 수 있습니다. 모든 삭제 또는 Put Object 보존 작업에는 가 포함되어야 합니다 `x-amz-bypass-governance-retention:true` 요청 헤더. 이러한 사용자는 다음과 같은 추가 작업을 수행할 수 있습니다.

- 개체 삭제 또는 여러 개체 삭제 작업을 수행하여 보존 기간이 경과하기 전에 개체 버전을 삭제합니다.

  법적 증거 자료 보관 중인 객체는 삭제할 수 없습니다. 법적 증거 자료 보관 기능을 해제해야 합니다.

- 오브젝트의 보존 기간이 경과하기 전에 오브젝트 버전의 모드를 거버넌스로부터 규정 준수로 변경하는 Put Object 보존 작업을 수행합니다.

  규정 준수 모드를 거버넌스로 변경하는 것은 허용되지 않습니다.

- 개체 유지 작업을 수행하여 개체 버전의 보존 기간을 늘리거나 줄이거나 제거합니다.

관련 정보

- "S3 오브젝트 잠금으로 오브젝트 관리"
- "S3 오브젝트 잠금을 사용하여 오브젝트를 보존합니다"
- "Amazon Simple Storage Service 사용자 가이드: S3 Object Lock 사용"

S3 라이프사이클 구성을 생성합니다

S3 라이프사이클 구성을 생성하여 StorageGRID 시스템에서 특정 오브젝트 삭제 시기를 제어할 수 있습니다.

이 섹션의 간단한 예는 S3 라이프사이클 구성에서 특정 S3 버킷에서 특정 객체가 삭제(만료)되는 시기를 제어하는 방법을 보여줍니다. 이 섹션의 예제는 설명을 위한 것입니다. S3 라이프사이클 구성 생성에 대한 자세한 내용은 를 참조하십시오 "Amazon Simple Storage Service 개발자 가이드: 개체 수명 주기 관리". StorageGRID는 만료 작업만 지원하며 전환 작업은 지원하지 않습니다.

문서 수정 상태 설정은 무엇입니까

라이프사이클 구성은 특정 S3 버킷의 오브젝트에 적용되는 규칙 세트입니다. 각 규칙은 영향을 받는 개체와 해당 개체가 만료되는 시기(특정 날짜 또는 특정 일 수 이후)를 지정합니다.

StorageGRID는 수명 주기 구성에서 최대 1,000개의 수명 주기 규칙을 지원합니다. 각 규칙에는 다음 XML 요소가 포함될 수 있습니다.

- 만료: 지정된 날짜에 도달하거나 지정된 일 수에 도달할 때 개체를 인제스트할 때로부터 개체를 삭제합니다.
- NoncurrentVersionExpiration: 지정된 일 수에 도달할 때 개체가 비전류가 되었을 때부터 개체를 삭제합니다.
- 필터(접두사, 태그)
- 상태
- ID입니다

버킷에 라이프사이클 구성을 적용하는 경우 버킷의 라이프사이클 설정은 항상 StorageGRID ILM 설정을 재정의합니다. StorageGRID는 ILM이 아닌 버킷의 만료 설정을 사용하여 특정 개체의 삭제 또는 유지 여부를 결정합니다.

따라서 ILM 규칙의 배치 지침이 개체에 계속 적용되더라도 그리드에서 개체를 제거할 수 있습니다. 또는 개체에 대한 ILM 배치 지침이 만료된 후에도 개체가 그리드에 남아 있을 수 있습니다. 자세한 내용은 을 참조하십시오 "ILM이 개체 수명 전반에 걸쳐 작동하는 방식".

> 버킷 수명 주기 구성은 S3 오브젝트 잠금이 활성화된 버킷과 함께 사용할 수 있지만 버킷 수명 주기 구성은 레거시 준수 버킷에서 지원되지 않습니다.

StorageGRID는 다음 버킷 작업을 사용하여 라이프사이클 구성을 관리합니다.

- 버킷 수명 주기를 삭제합니다
- 버킷 수명 주기 가져오기
- 버킷 수명 주기를 놓습니다

문서 수정 상태 설정 작성

라이프사이클 구성을 만드는 첫 번째 단계에서는 하나 이상의 규칙이 포함된 JSON 파일을 만듭니다. 예를 들어 이 JSON 파일에는 다음과 같은 세 가지 규칙이 포함되어 있습니다.

1. 규칙 1은 접두사와 일치하는 객체에만 적용됩니다 `category1/` 및 이(가) 있습니다 `key2` 의 값 `tag2`. 를 클릭합니다 `Expiration` 매개 변수는 필터와 일치하는 개체가 2020년 8월 22일 자정에 만료되도록 지정합니다.

2. 규칙 2는 접두사와 일치하는 객체에만 적용됩니다 `category2/`. 를 클릭합니다 `Expiration` 매개 변수는 필터와 일치하는 개체가 수집된 후 100일이 경과하도록 지정합니다.

> 일 수를 지정하는 규칙은 오브젝트가 수집된 시점을 기준으로 합니다. 현재 날짜가 수집 날짜와 일 수를 더한 값을 초과하면 라이프사이클 구성이 적용되는 즉시 일부 객체가 버킷에서 제거될 수 있습니다.

3. 규칙 3은 접두사와 일치하는 객체에만 적용됩니다 `category3/`. 를 클릭합니다 `Expiration` 매개 변수 일치하는 개체의 현재 버전이 아닌 버전이 최신 상태가 아닌 후 50일 후에 만료되도록 지정합니다.

```
{
    "Rules": [
        {
            "ID": "rule1",
            "Filter": {
                "And": {
                    "Prefix": "category1/",
                    "Tags": [
                        {
                            "Key": "key2",
                            "Value": "tag2"
                        }
                    ]
                }
            },
            "Expiration": {
                "Date": "2020-08-22T00:00:00Z"
            },
            "Status": "Enabled"
        },
        {
            "ID": "rule2",
            "Filter": {
                "Prefix": "category2/"
            },
            "Expiration": {
                "Days": 100
            },
            "Status": "Enabled"
        },
        {
            "ID": "rule3",
            "Filter": {
                "Prefix": "category3/"
            },
            "NoncurrentVersionExpiration": {
                "NoncurrentDays": 50
            },
            "Status": "Enabled"
        }
    ]
}
```

버킷에 라이프사이클 구성을 적용합니다

문서 수정 상태 구성 파일을 작성한 후 PUT Bucket 수명주기 요청을 실행하여 이를 버킷에 적용합니다.

이 요청은 예제 파일의 문서 수정 상태 구성을 이름이 인 버킷의 오브젝트에 적용합니다 `testbucket`.

```
aws s3api --endpoint-url <StorageGRID endpoint> put-bucket-lifecycle-
configuration
--bucket testbucket --lifecycle-configuration file://bktjson.json
```

라이프사이클 구성이 버킷에 성공적으로 적용되었는지 확인하려면 Get Bucket 수명주기 요청을 실행합니다. 예를 들면 다음과 같습니다.

```
aws s3api --endpoint-url <StorageGRID endpoint> get-bucket-lifecycle-
configuration
 --bucket testbucket
```

성공적으로 응답하면 방금 적용한 문서 수정 상태 설정이 나열됩니다.

버킷 수명 주기 만료가 객체에 적용되는지 확인합니다

Put Object, Head Object 또는 Get Object 요청을 실행할 때 수명 주기 구성의 만료 규칙이 특정 개체에 적용되는지 확인할 수 있습니다. 규칙이 적용될 경우 응답에는 가 포함됩니다 `Expiration` 객체가 만료되는 시간과 일치하는 만료 규칙을 나타내는 매개 변수입니다.

> 버킷 라이프사이클이 ILM, 을 무시하기 때문입니다 `expiry-date` 객체가 삭제될 실제 날짜가 표시됩니다. 자세한 내용은 을 참조하십시오 "개체 보존이 결정되는 방식".

예를 들어, 이 PUT 오브젝트 요청은 2020년 6월 22일에 발행되었으며 에 오브젝트를 두었습니다 `testbucket` 버킷.

```
aws s3api --endpoint-url <StorageGRID endpoint> put-object
--bucket testbucket --key obj2test2 --body bktjson.json
```

성공 응답은 개체가 100일(2020년 10월 1일) 내에 만료되고 라이프사이클 구성의 규칙 2와 일치함을 나타냅니다.

```
{
      *"Expiration": "expiry-date=\"Thu, 01 Oct 2020 09:07:49 GMT\", rule-
id=\"rule2\"",
      "ETag": "\"9762f8a803bc34f5340579d4446076f7\""
}
```

예를 들어, 이 head Object 요청은 testbucket 버킷에서 동일한 객체에 대한 메타데이터를 가져오는 데 사용되었습니다.

```
aws s3api --endpoint-url <StorageGRID endpoint> head-object
--bucket testbucket --key obj2test2
```

성공 응답에는 개체의 메타데이터가 포함되며 개체가 100일 후에 만료되고 규칙 2와 일치함을 나타냅니다.

```
{
      "AcceptRanges": "bytes",
      *"Expiration": "expiry-date=\"Thu, 01 Oct 2020 09:07:48 GMT\", rule-
id=\"rule2\"",
      "LastModified": "2020-06-23T09:07:48+00:00",
      "ContentLength": 921,
      "ETag": "\"9762f8a803bc34f5340579d4446076f7\""
      "ContentType": "binary/octet-stream",
      "Metadata": {}
}
```

S3 REST API 구현을 위한 권장사항

StorageGRID와 함께 사용할 S3 REST API를 구현할 때는 다음 권장 사항을 따라야 합니다.

존재하지 않는 객체에 대한 헤드 권장 사항

응용 프로그램에서 개체가 실제로 존재하지 않을 것으로 예상되는 경로에 개체가 있는지 정기적으로 확인하는 경우 ""사용 가능한"" 일관성 제어를 사용해야 합니다. 예를 들어, 응용 프로그램이 해당 위치에 배치되기 전에 위치를 지정할 경우 ""사용 가능"" 정합성 제어를 사용해야 합니다.

그렇지 않으면 헤드 작업에서 개체를 찾지 못할 경우 하나 이상의 스토리지 노드를 사용할 수 없는 경우 500개의 내부 서버 오류가 발생할 수 있습니다.

PUT Bucket 정합성 보장 요청을 사용하여 각 버킷에 대해 ""사용 가능" 정합성 제어를 설정하거나 개별 API 작업에 대한 요청 헤더에서 정합성 제어를 지정할 수 있습니다.

개체 키에 대한 권장 사항

버킷이 처음 생성된 시점을 기준으로 오브젝트 키 이름에 대한 다음 권장 사항을 따르십시오.

StorageGRID 11.4 또는 이전 버전에서 생성된 버킷

- 개체 키의 처음 네 문자로 임의 값을 사용하지 마십시오. 이는 이전 AWS에서 권장하는 키 접두사와 다릅니다. 대신 와 같이 고유하지 않은 접두어를 사용합니다 `image`.
- 이전 AWS 권장 사항에 따라 키 접두사에 랜덤 및 고유 문자를 사용하려면 오브젝트 키에 디렉토리 이름이 접두사로 지정됩니다. 즉, 다음 형식을 사용합니다.

  `mybucket/mydir/f8e3-image3132.jpg`

  이 형식 대신:

```
mybucket/f8e3-image3132.jpg
```

StorageGRID 11.4 이상에서 생성된 버킷

성능 모범 사례에 맞게 개체 키 이름을 제한하는 것은 필요하지 않습니다. 대부분의 경우 개체 키 이름의 처음 4개 문자에 임의의 값을 사용할 수 있습니다.

> 단, 짧은 시간 내에 모든 오브젝트를 지속적으로 제거하는 S3 워크로드가 예외입니다. 이 사용 사례에 대한 성능 영향을 최소화하려면 키와 같은 1000개의 오브젝트마다 주요 이름의 앞부분을 다르게 지정해야 합니다. 예를 들어, S3 클라이언트가 일반적으로 초당 2,000개의 오브젝트를 기록하고 ILM 또는 버킷 라이프사이클 정책에 따라 3일 후에 모든 오브젝트를 제거한다고 가정해 보겠습니다. 성능에 미치는 영향을 최소화하기 위해 다음과 같은 패턴을 사용하여 키의 이름을 지정할 수 있습니다.
> `/mybucket/mydir/yyyymmddhhmmss-random_UUID.jpg`

""범위 읽기" 권장 사항

를 누릅니다 "저장된 개체를 압축하는 전역 옵션" 이 활성화되면 S3 클라이언트 응용 프로그램은 반환할 바이트 범위를 지정하는 Get Object 작업을 수행하지 않아야 합니다. 이러한 ""범위 읽기"" 작업은 StorageGRID가 요청된 바이트에 액세스하기 위해 개체의 압축을 효과적으로 해제해야 하기 때문에 비효율적입니다. 매우 큰 개체에서 작은 바이트 범위를 요청하는 Get Object 작업은 특히 비효율적입니다. 예를 들어, 50GB의 압축 개체에서 10MB 범위를 읽는 것은 비효율적입니다.

압축된 개체에서 범위를 읽으면 클라이언트 요청이 시간 초과될 수 있습니다.

> 개체를 압축해야 하고 클라이언트 응용 프로그램에서 범위 읽기를 사용해야 하는 경우 응용 프로그램의 읽기 시간 초과를 늘리십시오.

관련 정보

- "일관성 제어"
- "버킷 일관성을 유지합니다"
- "StorageGRID 관리"

저작권 정보

Copyright © 2023 NetApp, Inc. All Rights Reserved. 미국에서 인쇄된 본 문서의 어떠한 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 수단(복사, 녹음, 녹화 또는 전자 검색 시스템에 저장하는 것을 비롯한 그래픽, 전자적 또는 기계적 방법)으로도 복제될 수 없습니다.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 '있는 그대로' 제공되며 상품성 및 특정 목적에의 적합성에 대한 명시적 또는 묵시적 보증을 포함하여(이에 제한되지 않음) 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 영업 중단을 포함하여(이에 국한되지 않음), 이 소프트웨어의 사용으로 인해 발생하는 모든 직접 및 간접 손해, 우발적 손해, 특별 손해, 징벌적 손해, 결과적 손해의 발생에 대하여 그 발생 이유, 책임론, 계약 여부, 엄격한 책임, 불법 행위(과실 또는 그렇지 않은 경우)와 관계없이 어떠한 책임도 지지 않으며, 이와 같은 손실의 발생 가능성이 통지되었다 하더라도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구매의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허, 해외 특허 또는 출원 중인 특허로 보호됩니다.

제한적 권리 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.227-7013(2014년 2월) 및 FAR 52.227-19(2007년 12월)의 기술 데이터-비상업적 품목에 대한 권리(Rights in Technical Data -Noncommercial Items) 조항의 하위 조항 (b)(3)에 설명된 제한사항이 적용됩니다.

여기에 포함된 데이터는 상업용 제품 및/또는 상업용 서비스(FAR 2.101에 정의)에 해당하며 NetApp, Inc.의 독점 자산입니다. 본 계약에 따라 제공되는 모든 NetApp 기술 데이터 및 컴퓨터 소프트웨어는 본질적으로 상업용이며 개인 비용만으로 개발되었습니다. 미국 정부는 데이터가 제공된 미국 계약과 관련하여 해당 계약을 지원하는 데에만 데이터에 대한 전 세계적으로 비독점적이고 양도할 수 없으며 재사용이 불가능하며 취소 불가능한 라이센스를 제한적으로 가집니다. 여기에 제공된 경우를 제외하고 NetApp, Inc.의 사전 서면 승인 없이는 이 데이터를 사용, 공개, 재생산, 수정, 수행 또는 표시할 수 없습니다. 미국 국방부에 대한 정부 라이센스는 DFARS 조항 252.227-7015(b)(2014년 2월)에 명시된 권한으로 제한됩니다.

상표 정보

NETAPP, NETAPP 로고 및 http://www.netapp.com/TM에 나열된 마크는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.